

Fundamentals of Boolean Algebra

3.1.1 Introduction to Boolean Algebra

A digital computer processes the data which is in the form of 0's and 1's. Processing involves performing operations on bits. Operations include addition, subtraction, magnitude comparison, code conversion, etc. A computer consists of several logic circuits, which will help process the data. Each logic circuit represents one or more logic functions that are implemented using logic circuit elements. Boolean algebra is one of the techniques that are useful while simplifying and analyzing logic functions.

Boolean algebra is the branch of algebra in which the values of the variables are the truth values. It is much simpler than general algebra as it deals with only two binary bits, 0 and 1. Boolean algebra is also known as the algebra of logic or switching algebra. This well-known algebra deals with the rules by which logical operations are carried out.

Boolean algebra was developed by a famous British Mathematician, George Boole. He developed a set of rules for the manipulations of binary variables. Boolean algebra is the basis for all digital systems simplification, such as digital computers, calculators, embedded systems, etc. In such systems, a digital circuit is represented by a set of input and output symbols, and the function is expressed as a set of Boolean relationships between these symbols. Boolean algebra is supported by several postulates and theorems that help simplify Boolean functions. Elementary algebra is expressed using basic mathematical operations, such as addition, subtraction, multiplication, and division, whereas the basic operations of Boolean algebra are conjunction, disjunction, and negation. These Boolean operations are expressed with the corresponding binary operators AND, OR, and the unary operator NOT, and these are collectively referred to as Boolean operators.

3.1.2 Important Boolean Algebra Terms

The following are terms that are used in Boolean algebra.

Boolean Value

Boolean variables hold values that are known as Boolean values or truth values. The Boolean value can be either true or false. In binary notation. Binary number 1 denotes TRUE value, whereas binary 0 denotes FALSE value. The Boolean function describes how the Boolean output is derived from Boolean inputs. Arguments and results of a Boolean function assume values from a two-element set, zero and one.

For example,

$$F(X,Y) = X'Y$$

$F(X, Y)$ is a function of two Boolean variables X and Y , and X, Y can assume values 0 or 1. Since there are two variables, four combinations of inputs are possible. Based on the input values, function output will be obtained. The right-hand side of the Boolean function is the Boolean expression.

Boolean expression is a logical statement that can yield either a TRUE or False value. Boolean expression is obtained by combining binary variables, the constants 0 and 1, and logical operators, AND, OR, and NOT operators.

For example, $F(X, Y) = 1 + X \cdot Y + Y'$

where $F(X, Y)$ is the Boolean function with two variables X and Y . 1 is a constant. Dot is an AND operator. Plus is an OR operator. And finally, a dash is a NOT operator. Usually, the dot is not written while writing the Boolean expression.

Truth Table

A truth table is a table that shows the relationship between inputs and outputs.

For example, $F(X, Y)$ is a function with two variables, X and Y , and the Boolean expression is given by

$$F(X, Y) = XY + Y'$$

The truth table for this expression is shown in Figure 1. Since there are two variables, four input combinations are possible, that is, 00, 01, 10, and 11. The function produces Logic 1 only when X AND Y is 1 or Y' is one.

Inputs		Output $F(X, Y)$
X	Y	$XY + Y'$
0	0	$00 + 1 \rightarrow 1$
0	1	$01 + 0 \rightarrow 0$
1	0	$10 + 1 \rightarrow 1$
1	1	$11 + 0 \rightarrow 1$

Figure 1: Truth table for the function $F(X, Y) = XY + Y'$.

The Rule of Precedence

Consider the following example, where Boolean expression is expressed using three variables X , Y , and Z .

$$F(X, Y, Z) = XY + Y(X' + Z)$$

The expression contains multiple operators. Left to right evolution involves evaluating subexpression X dot Y first and then OR it with Y , then subexpression within brackets, $X' + Z$. Finally, AND with previous results. Another way is to begin from the right side. Perform $X' + Z$, then AND with Y . Then evaluate XY . Finally, perform OR operation. Sometimes both methods may yield the same output, sometimes not. In general mathematics, the BODMAS rule is used while simplifying mathematical expressions. In Boolean algebra, the precedence rule is also needed to avoid ambiguity in the simplification. Operator precedence helps to determine which of the operators in an expression must be evaluated first in case the expression consists of more than a single operator. Following is the precedence order:

1. Brackets
2. NOT
3. AND
4. OR

Tautology and Fallacy

If the result of a Boolean expression is always TRUE or 1, it is called a tautology. If the result of a Boolean expression is always FALSE or 0, it is called a fallacy.

Postulates and Theorems

3.2.1 Postulates of Boolean Algebra: Part 1

Generally, algebra is supported by many theorems and postulates. While proving theorems, certain assumptions are made. These assumptions are known as postulates. A postulate is a statement that is assumed to be true without proof. In Boolean algebra also, there are postulates or laws and theorems.

Logical inverse refers to complementation. The complement of 0 is 1, and 1 is 0. Logically, it is represented as an inverter, which is shown in Figure 1.

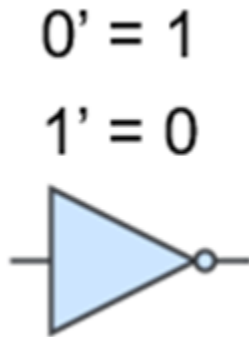


Figure 1: Logic inverse.

In general, duality is a principle using which one can obtain a true statement from another by merely interchanging two words. For example, the sentence: “Eat to live.” By interchanging the words, eat and live, new sentence is obtained, “Live to eat,” the meaning of which is different from “Eat to live.” Similarly, in Boolean algebra, using the principle of duality, dual of the Boolean function is obtained by interchanging the logical OR operator with the logical AND operator and zeros with ones. For example, the Boolean expression $X + 1 = 1$. This expression $X + 1$ always yields one irrespective of the value of x . The dual of this expression can be obtained by replacing OR with AND, 1 with 0. So, the expression will be $X \cdot 0 = 0$.

There are four postulates or laws used in Boolean algebra.

- Identity postulate
- Complement postulate
- Commutative postulate
- Distributive postulate

Every postulate has two laws based on the principle of duality.

Identity Postulate

The identity postulate has two expressions. The first expression is $X + 0 = X$.

That is, the sum of a variable, and 0 is the variable itself. This expression is implemented using an OR gate. This postulate can be proved using a Truth table shown in Figure 2.

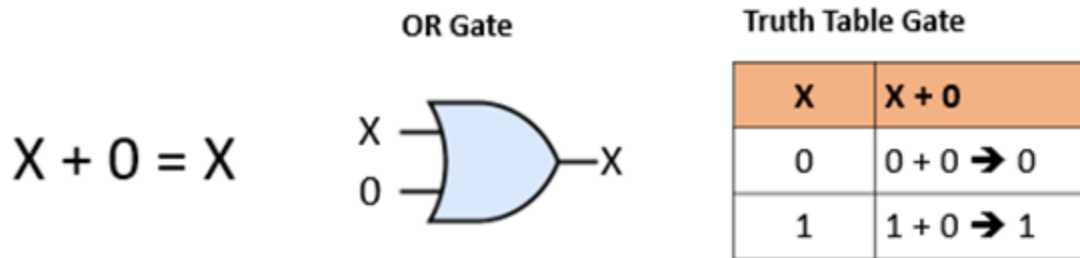


Figure 2: Identity postulate – $X + 0 = X$.

When $X = 0$, $0 + 0$ is equal to 0, and when $X = 1$, $1 + 0$ will be 1. As you can see, the output is equal to input X .

The second expression is obtained by applying the principle of duality

$$X \cdot 1 = X.$$

This expression can be implemented using AND gate. The expression can be verified using a truth table, shown in Figure 3. When $X = 0$, $0 \cdot 1$ is 0, whereas when $X = 1$, $1 \cdot 1$ is 1. As you can see, the output equals input, X .

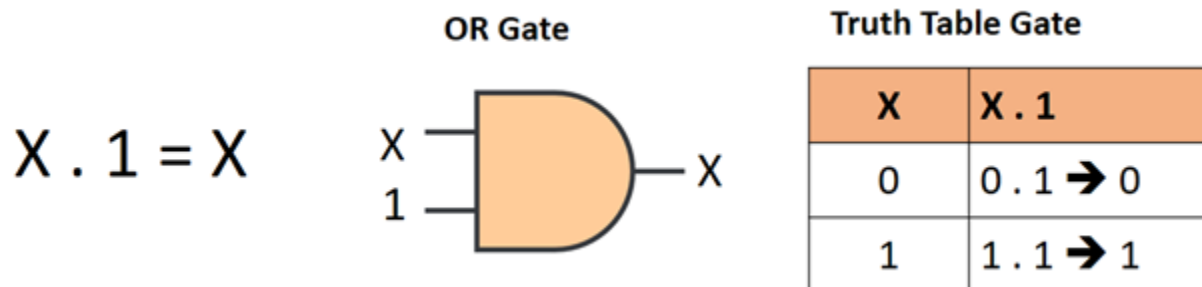


Figure 3: Identity postulate – $X \cdot 1 = X$.

3.2.2 Postulates of Boolean Algebra: Part 2

Complement Postulate:

The complement postulate states that ORing a variable with its complement always yields a logic 1, that is,

$$X + X' = 1$$

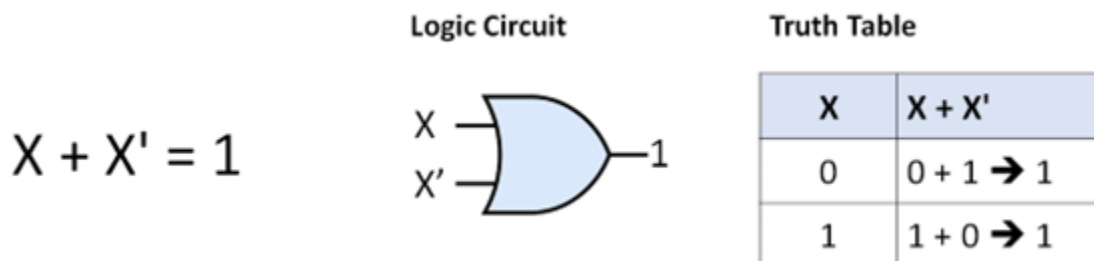


Figure 4: Complement postulate – $X + X' = 1$.

The logical diagram and truth table are shown in Figure 4. This law is implemented using an OR gate. When X equals 0, since X' is 1, the output will be 1. When X equals 1, the output is 1.

Therefore, irrespective of the value of X , the output is always 1.

By applying the principle of duality, another law is obtained and given by

$$X \cdot X' = 0$$

This law is implemented using AND gate. The logic diagram and truth table for this dual expression are shown in Figure 5. As indicated in the truth table, irrespective of input X's value, the output is zero.

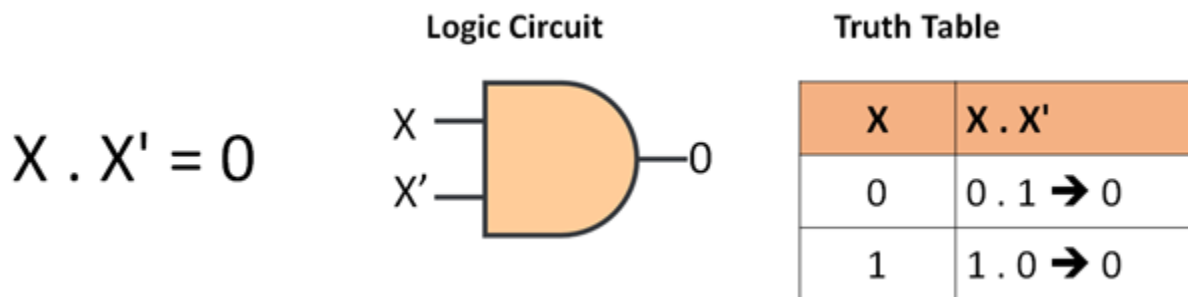


Figure 5: Complement postulate – $X \cdot X' = 0$.

Commutative Postulate

The commutative postulate states that interchanging the order of operands in a Boolean equation does not change its result. For example,

$$X + Y = Y + X.$$

Both LHS and RHS expressions yield the same output. The logic circuit for LHS expression and RHS expression is shown in Figure 6. Both expressions can easily be verified using the truth table. When $X = 0$ and $Y = 0$, both expressions yield 0. When either of the input is one or both inputs are 1, the output is 1.

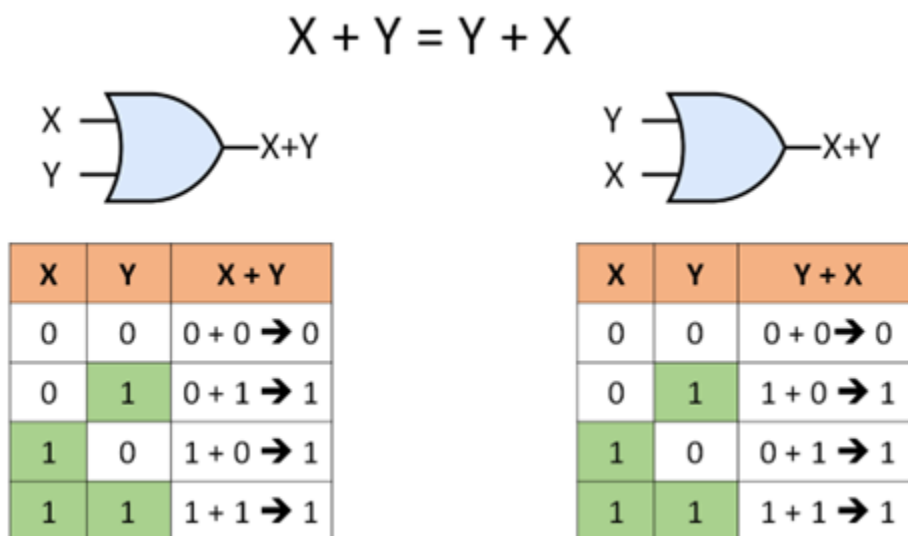


Figure 6: Commutative law – $X + Y = Y + X$.

By changing OR to AND, another law is obtained and is given by,

$$X \cdot Y = Y \cdot X$$

Here, LHS and RHS expressions are implemented using AND gate, which is shown in Figure 7.

When one of the inputs or both of the inputs are 0, the output is 0. When both the inputs are one, the output is 1. This is valid for both LHS and RHS expressions.

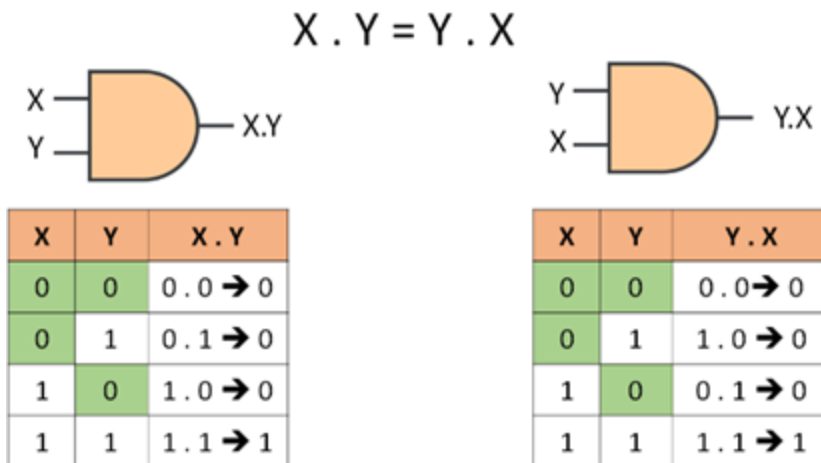


Figure 7: Commutative law – $X \cdot Y = Y \cdot X$.

Example 1: Prove that $AB + BC + B'C = AB + C$

Solution:

$$\text{LHS} \square AB + BC + B'C$$

$$\square AB + C (B + B')$$

$$\square AB + C \cdot 1 [B + B' = 1]$$

$$\square AB + C [C \cdot 1 = C]$$

Example 2: Simplify the expression $x'y + xy + x'y' + y'x$

Solution:

$$\text{LHS} \square x'y + xy + x'y' + y'x$$

$$\square y(x' + x) + x'y' + y'x$$

$$\square y(1) + x'y' + y'x [x' + x = 1]$$

$$\square y + x'y' + y'x [y \cdot 1 = y]$$

$$\square y + y'(x' + x)$$

$$\square y + y'(1) [x' + x = 1]$$

$$\square y + y'[y' \cdot 1 = y']$$

$$\square 1[y + y' = 1]$$

3.2.3 Postulates of Boolean Algebra: Part 3

Distributive Postulate

The distributive law states that ORing several variables and ANDing the result with a single variable is equivalent to ANDing the result with a single variable with each of the several variables and then ORing the products. For example,

$$X \cdot (Y + Z) = X \cdot Y + X \cdot Z$$

As can be seen from Figure 8, the left-hand side of the expression is implemented using one OR gate and one AND gate, whereas the right-hand side of the expression is implemented using two AND gates and one OR gate.

$$X \cdot (Y + Z) = X \cdot Y + X \cdot Z$$

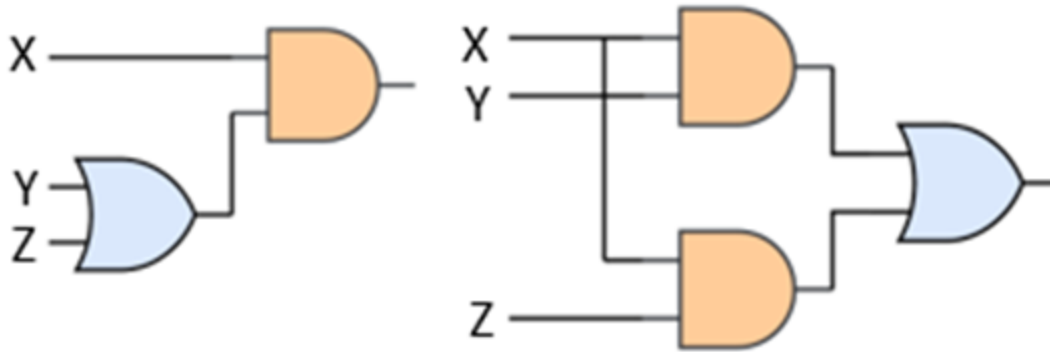


Figure 8: Distributive first law.

The second one is obtained by applying the principle of duality. That is, by changing OR to AND, AND to OR. The second postulate and its logic implementation are shown in Figure 9.

$$X + (Y \cdot Z) = (X + Y) \cdot (X + Z)$$

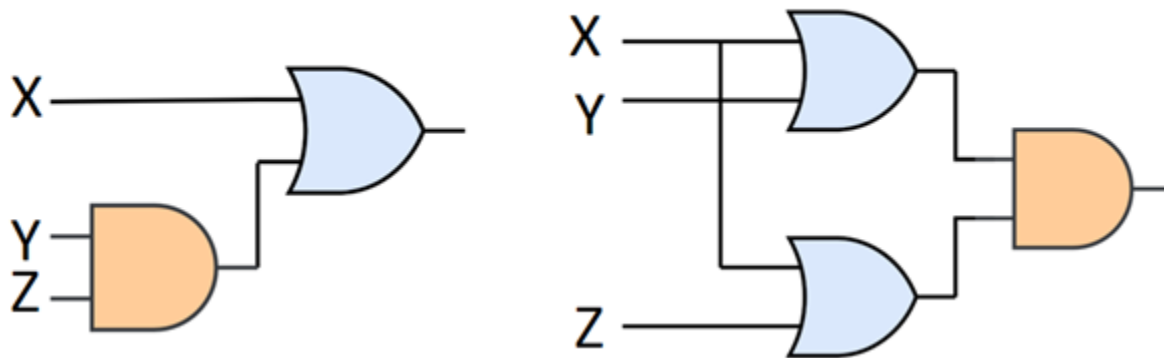


Figure 9: Distributive second law.

3.2.4 Boolean Algebra Theorems: Part 1

Idempotent Theorem

The idempotent first law states that adding a variable to the same variable will result in the same variable.

$$X + X = X$$

The logical diagram in Figure 10 shows how to visualize this law. The implementation uses an OR gate.

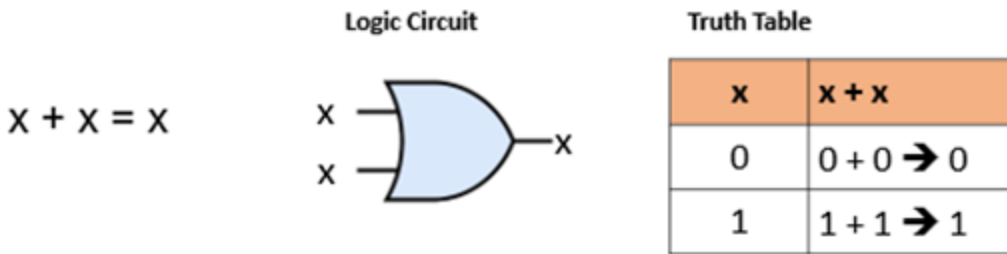


Figure 10: Idempotent first law.

When X equals 0, zero plus zero will be equal to zero. When X equals 1, one plus one will be equal to one. So, X plus X equals X itself. By applying the principle of duality, the second idempotent Law is obtained and is given by

$$X.X = X$$

The second idempotent law states that ANDing a variable with itself yields the variable itself. AND gate exhibits idempotence. Figure 11 shows the logic implementation and the truth table. When X equals 0, zero AND zero is equal to zero. When X equals 1, one AND one is equal to one. So, X AND X equals X itself.

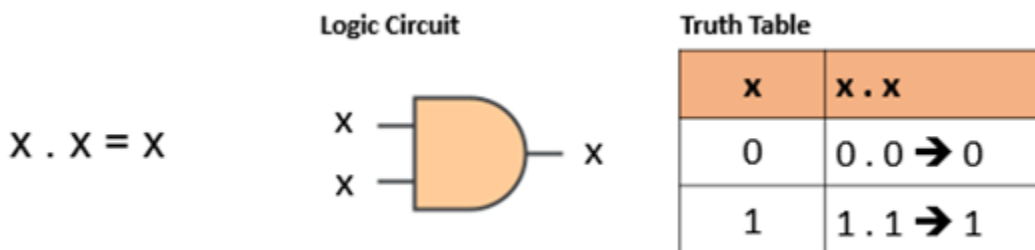


Figure 11: Idempotent second law.

Annulment Theorem

The first law of annulment states that a variable that is ORed with 1 gives a 1. That is, $X+1 = 1$.

Logic implementation and truth table are shown in Figure 12. When X equals 0, zero OR one is equal to one, and when X equals 1, one OR one will be equal to one. Hence, X OR 1 equals one.

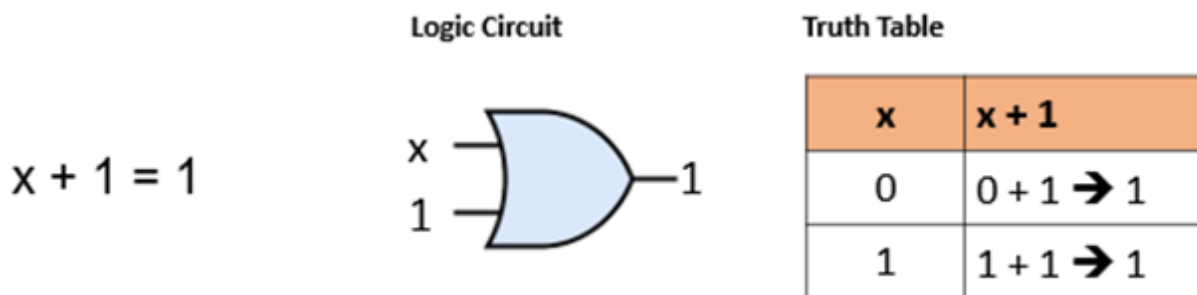


Figure 12: Annulment first law.

By applying the principle of duality, that is, changing OR to AND, and 1 to 0, one more law of annulment is obtained.

$$X.0 = 0$$

Logic implementation and truth table are shown in Figure 13.

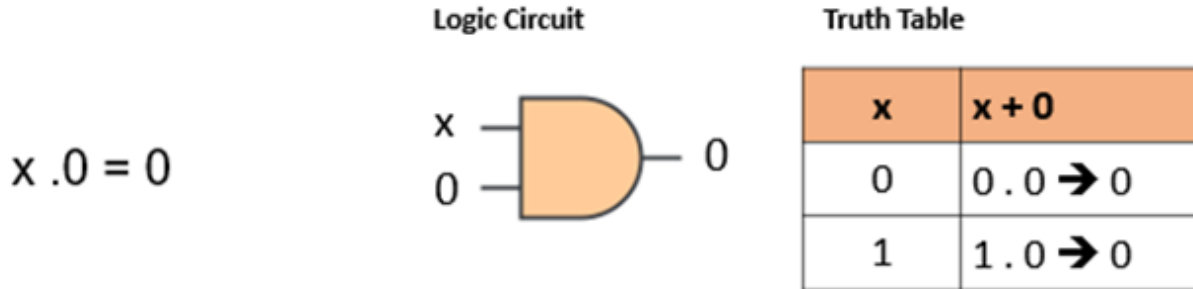


Figure 13: Annulment second law.

Involution Theorem

Involution law is also known as double complement law. This law states that the double complement of a variable gives the same variable. That is,

$$(X')' = X$$

Logic circuit implementation and truth table are shown in Figure 14. When X equals zero, X' is one and (x')' is zero. Similarly, when X equals one, X' is zero and (x')' one. Hence, (X')' equals X.

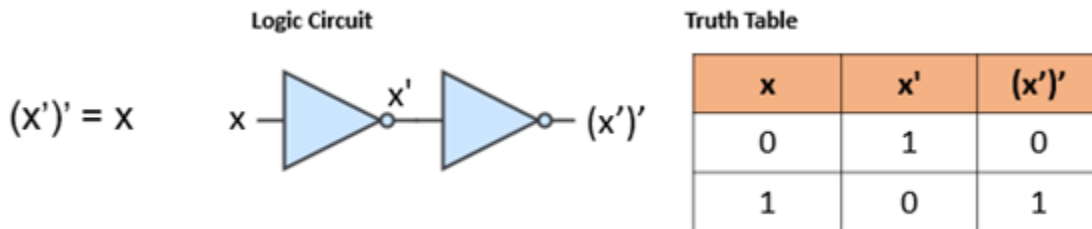


Figure 14: Involution law.

3.2.5 Boolean Algebra Theorems: Part 2

Associative Theorem

The associative theorem allows the removal of brackets from an expression and regrouping of the terms. This theorem states that the operation can be performed in any order when the variable's priorities are the same.

There are two laws under this:

1. OR associative law
2. AND associative law

One law can be obtained by applying the principle of duality to the other law. OR associative law is also known as the law of addition.

For three variables X, Y, and Z, OR associativity is written as

$$X + (Y + Z) = (X + Y) + Z$$

The logic implementation of LHS and RHS is shown in Figure 15. These two implementations yield the same output for different combinations of input variables, which can be verified using the truth table method.

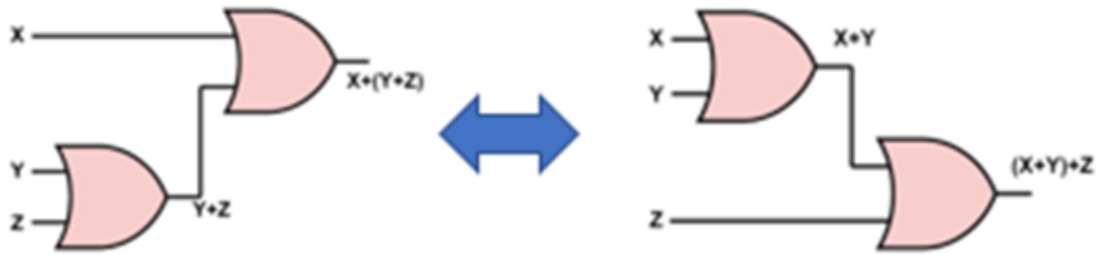


Figure 15: OR associativity law.

By applying the principle of duality, AND associative law is obtained. AND associative law is also known as the law of multiplication. For three variables, it can be written as

$$X(YZ) = (XY)Z$$

The logic implementation of LHS and RHS is shown in Figure 16. Here also, both implementations yield the same output for different input combinations.

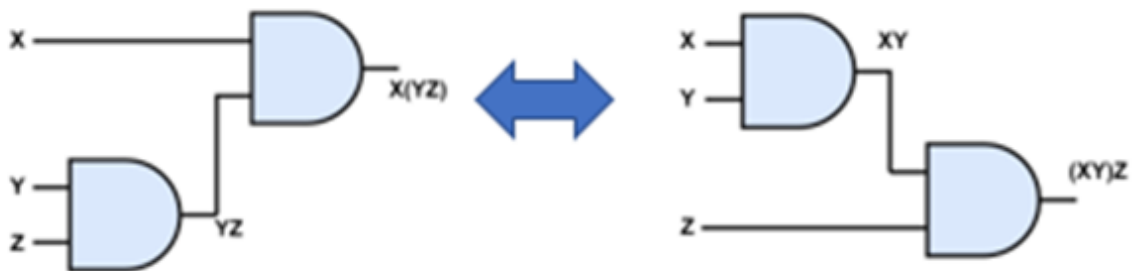


Figure 16: AND associativity law.

Absorption Theorem

This theorem enables a reduction in a complicated expression to a simpler one by absorbing like terms. Here is the first theorem under the absorption theorem.

$$X + XY = X$$

Proof:

$$\text{LHS} \square X + XY$$

$$\square X(1 + Y)$$

$$\square X \cdot 1 [1 + Y = 1 \text{ annulment law}]$$

$$\square X[X \cdot 1 = X]$$

By applying the principles of duality, another absorption theorem is obtained and is given by,

$$X(X + Y) = X$$

Proof:

$$\text{LHS} \square X(X + Y)$$

$$\square XX + XY [\text{Apply Distributive Law}]$$

$$\square X + XY [XX \square X]$$

$$\square X [\text{Apply Absorption Law}]$$

3.2.6 Boolean Algebra Theorems: Part 3

DeMorgan's Theorem

Augustus DeMorgan, a famous English mathematician, invented the two most popular theorems, which are used as powerful tools in digital design. DeMorgan's first theorem states that when two or

more input variables are ANDed and complemented, they are equivalent to the OR of the complements of the individual variables. Consider two variables, X and Y, then

$$(X.Y)' = X' + Y'$$

Logic gate circuit implementation is shown in Figure 17. The left-hand side logic expression can be implemented using a AND-NOT logic gates or using a standard NAND gate with inputs X and Y. The right-hand side gate arrangement first inverts the two inputs producing X' and Y' . These then become the inputs to the OR gate. Therefore, the output from the OR gate becomes $X' + Y'$. So, NAND gate is equivalent to bubbled OR gate. The truth table proves that both left-hand and right-hand expressions yield the same output for different input combinations.



Inputs		Outputs for Each term				
X	Y	XY	$(XY)'$	X'	Y'	$X' + Y'$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

Figure 17: DeMorgan's first theorem.

According to the second theorem, when two or more input variables are ORed and complemented, they are equivalent to AND of the complements of the individual variables.

$$(X + Y)' = X'.Y'$$

The logic circuit implementation and truth table are shown in Figure 18. The left-hand side logic expression can be implemented using an OR NOT logic gate or using a standard NOR gate with inputs X and Y. The right-hand side gate arrangement first inverts the two inputs producing X' and Y' . These then become the inputs to the AND gate. Therefore, the output from the AND gate becomes $X'.Y'$. The truth table proves that both left-hand and right-hand expressions yield the same output for different input combinations.



Inputs		Outputs for Each term				
X	Y	X+Y	$(X+Y)'$	X'	Y'	$X'.Y'$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Figure 18: De Morgan's second theorem.

Summary of Boolean Algebra – Postulates and Theorems

The list of postulates and theorems is tabulated in two tables.

Postulate #	Postulate Equation
1 Identity	(a) $x + 0 = x$
	(b) $x \cdot 1 = x$
2 Complement	(a) $x + x' = 1$
	(b) $x \cdot x' = 0$
3 Commutative	(a) $x + y = y + x$
	(b) $x \cdot y = y \cdot x$
4 Distributive	(a) $x \cdot (y + z) = x \cdot y + x \cdot z$
	(b) $x + (y \cdot z) = (x + y) \cdot (x + z)$

Theorem #	Theorem
1 Idempotent	(a) $x + x = x$
	(b) $x * x = x$
2 Annulment	(a) $x + 1 = 1$
	(b) $x * 0 = 0$
3 Involution	$(x')' = x$
4 Associative	(a) $x + (y + z) = (x + y) + z$
	(b) $x(yz) = (xy)z$
5 DeMorgan's	(a) $(x + y)' = x'y'$
	(b) $(xy)' = x' + y'$
6 Absorption	(a) $x + xy = x$
	(b) $x(x+y) = x$

3.2.7: Boolean Algebra: Exercise 1

Exercise 1: Simplify the following Boolean expression to a minimum number of literals
 $XY + XY'$

Solution:

- ☐ $XY + XY'$ [Apply Distributive Law]
- ☐ $X(Y+Y')$ [Apply Complement Law]
- ☐ $X(1)$ [Apply Identity Law]
- ☐ X

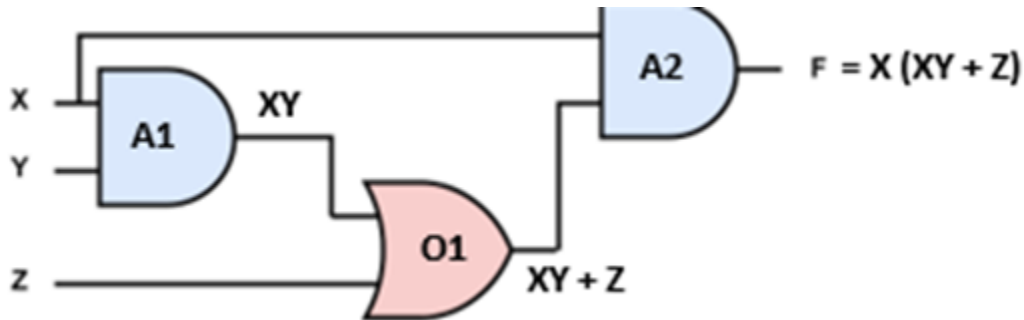
Exercise 2: Simplify the following Boolean expression to a minimum number of literals
 $(X+Y)(X+Y')$

Solution:

- ☐ $(X+Y)(X+Y')$ [Apply Distributive Law]
- ☐ $XX + XY' + YX + YY'$ [Apply Idempotent and Complement Law]
- ☐ $X + XY' + YX$ [Apply Commutative Law]
- ☐ $X + XY' + YX$ [Apply Distributive Law]
- ☐ $X(1 + Y' + Y)$ [Apply Annulment Law]
- ☐ X

3.2.8: Boolean Algebra: Exercise 2

Exercise 1: Use Boolean algebra to simplify the following logic circuit.



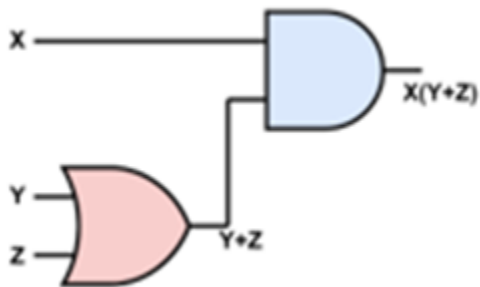
Solution:

$F = X(XY + Z)$ [Apply Distributive Law]

$\square XXY + XZ$ [Apply Idempotent Law]

$\square XY + XZ$ [Apply Distributive Law]

$\square X(Y + Z)$



Exercise 2: Reduce the following Boolean expression to five literals.

$ABC + A'B'C + A'BC + ABC' + A'B'C'$

Solution:

$\square ABC + ABC' + A'B'C + A'B'C' + A'BC$ [Apply Distributive Law(a)]

$\square AB(C+C') + A'B'(C+C') + A'BC$ [Apply Complement Law(a)]

$\square AB(1) + A'B'(1) + A'BC$ [Apply Identity Law(b)]

$\square AB + A'B' + A'BC$ [Apply Distributive Law(a)]

$\square AB + A'(B' + BC)$ [Apply Distributive Law(b)]

$\square AB + A'(B' + B)(B' + C)$ [Apply Complement Law(a)]

$\square AB + A'(1)(B' + C)$ [Apply Identity Law(b)]

$\square AB + A'(B' + C)$

3.2.9: Boolean Algebra: Exercise 3

Universal Gates:

A universal gate is a gate that can implement any Boolean function without the need to use any other gate type. NAND and NOR gates are known as universal gates. NOR and NAND gate implementation of NOT, OR, and AND gates is shown in Figure 19.

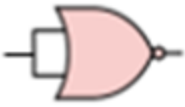
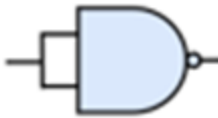
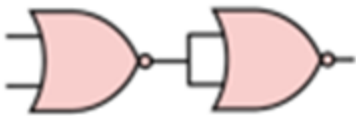
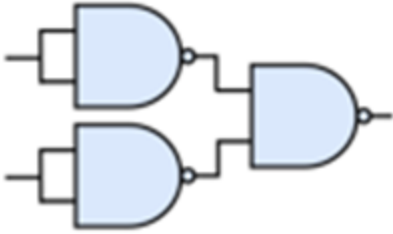
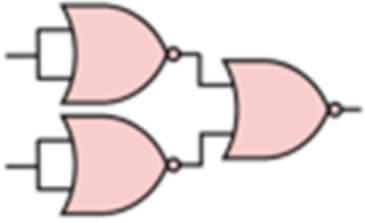
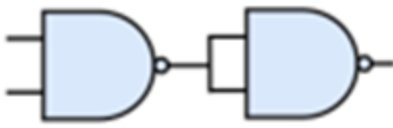
Gate	NOR Implementation	NAND Implementation
NOT		
OR		
AND		

Figure 19: Universal gate – NOR and NAND gates.

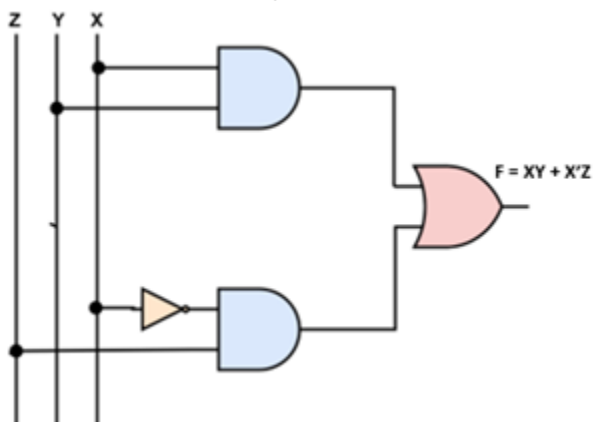
Exercise: Given the Boolean function:

$$F = XY + X'Z$$

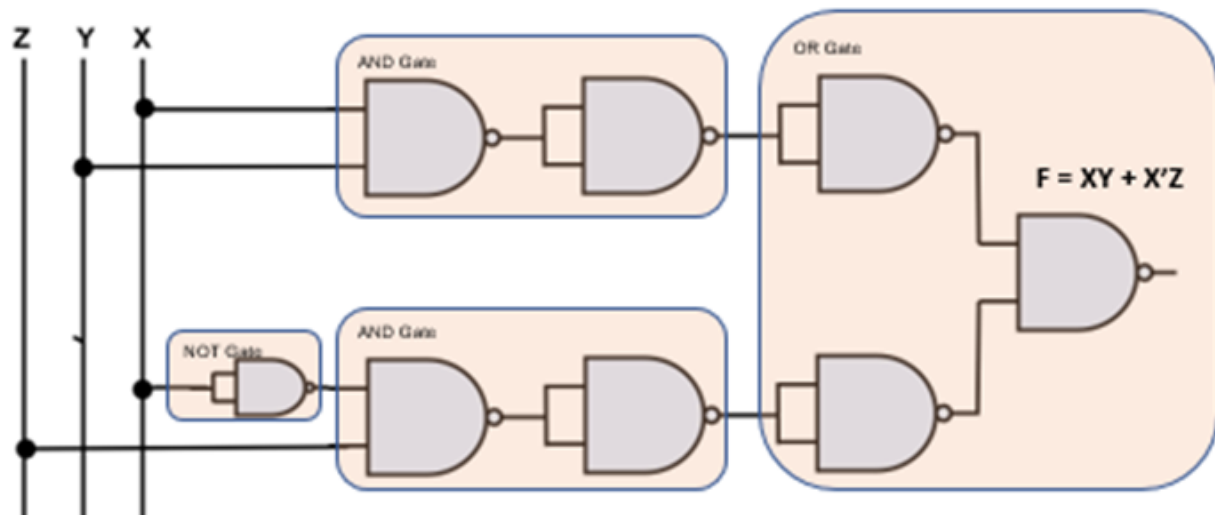
- Implement it with AND, OR, and NOT gates
- Implement it with only NAND gates
- Implement it with only NOR gates

Solution:

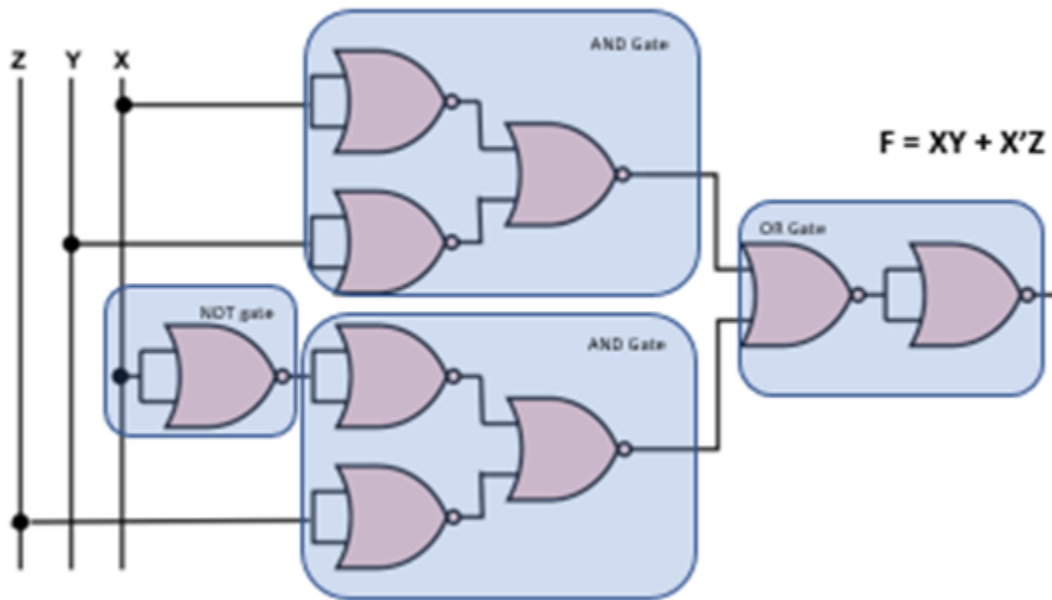
- AND, OR, and NOT gate implementation



- NAND gate implementation



c) NOR gate implementation



IC requirement is as follows:

Problem #	Implementation	IC requirement
a)	AND (IC 7408 – Quad 2 input AND gate) OR (IC 7432 - Quad 2 input AND gate) NOT (IC 7404 – Hex Inverter)	3
b)	NAND (IC 7400 – Quad 2 input NAND gate)	2
c)	NOR (IC 7402 – Quad 2 input NOR gate)	3

Boolean Functions: SOP and POS

3.3.1: Boolean Functions: Standard and Canonical Forms

Logical functions are expressed in terms of logical variables. The values assumed by the logical functions and the logical variables are in binary form. There are two ways of expressing Boolean expressions.

1. Sum of products form (SOP)
2. Product of sums form (POS)

Consider the logic circuit given in Figure 1. The logic circuit output is given by $F = WX + YZ$

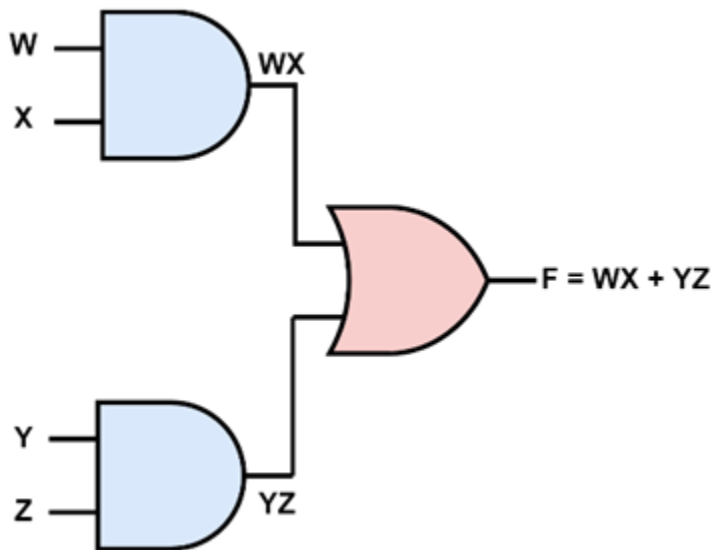


Figure 1: AND –OR logic circuit implementation.

The logic circuit shown is called an AND-OR circuit because input AND gates drive an output OR gate. The output equation is referred to as a sum-of-products equation. Note that the AND-OR network of gates always produces the sum of products, in short, known as SOP. The terms in SOP expression are known as minterms.

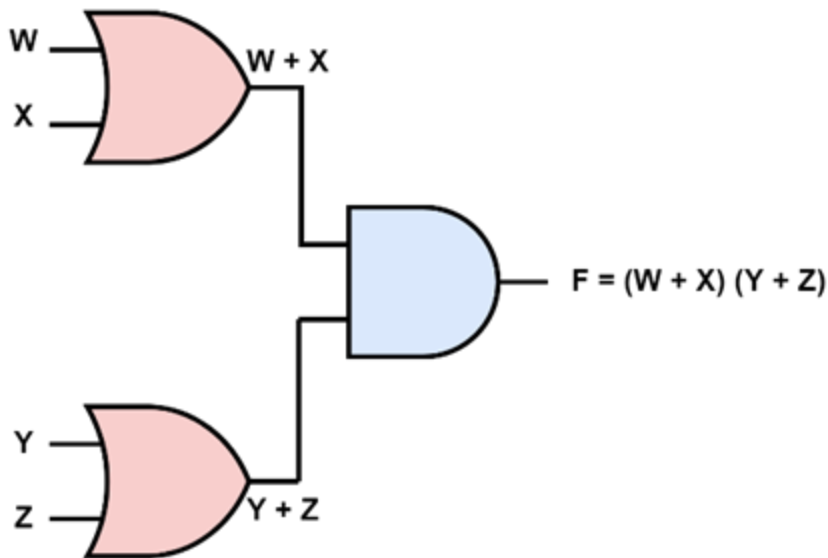


Figure 2: OR-AND logic circuit implementation.

The logic circuit shown in Figure 2 is called an OR-AND circuit because an output AND gate is driven by input OR gates. Note that the final output is the product of sums, in short, known as POS. OR-AND circuits always produce product of sums. The terms in POS expression are known as maxterms.

Canonical Form and Standard Form

In canonical form, each term of Boolean expression contains all input variables either in normal form or in complement form. For example:

$$F(X,Y) = XY + XY'$$

Both terms contain all the variables. On the other hand, standard form terms can contain one variable, two variables, or any number of variables in normal and complement form. For example:

$$F(X,Y) = X + XY'$$

Here, $F(X,Y)$ contains two terms. The first term has one variable and the second term has two variables.

3.3.2: Deriving SOP and POS Expression from a Truth Table

There are two ways of representing Boolean expressions.

- Sum of products, SOP expression
- Product of sums, POS expression

Consider a three-variable truth table shown in Figure 3, with three variables, X, Y, and Z, and can have eight combinations.

X	Y	Z	f	Minterm
0	0	0	0	m0 $\rightarrow$ X'Y'Z'
0	0	1	1	m1 $\rightarrow$ X'Y'Z
0	1	0	0	m2 $\rightarrow$ X'YZ'
0	1	1	1	m3 $\rightarrow$ X'YZ
1	0	0	0	m4 $\rightarrow$ XY'Z'
1	0	1	0	m5 $\rightarrow$ XY'Z
1	1	0	1	m6 $\rightarrow$ XYZ'
1	1	1	1	m7 $\rightarrow$ XYZ

Figure 3: Truth table for three variable SOP function.

Each combination represents one product term. For example, when $X = 0$, $Y = 0$, $Z = 1$, the product term is $X'Y'Z$. The eight product terms are known as minterms and are denoted as m0, m1, m2 to m7. Note that minterms are denoted by lowercase “m.” Two-variable truth tables will have four minterms, whereas four-variable truth tables will have 16 minterms.

The sum of product expression for the given truth table is

$$f = X'Y'Z + X'YZ + XYZ' + XYZ.$$

Sometimes Boolean expression is expressed in terms minterms and is given by

$$f = m1 + m3 + m6 + m7$$

$$\square \sum m(1, 3, 6, 7)$$

Figure 4 shows the product of sums expression for the given truth table. The product of sums expression is one in which different “sum” terms from inputs are “ANDed” together. A variable whose value is zero is represented in normal form, and if its value is one, it is represented in complement form. That is, if the value of X is zero, then it is represented as X and if the value of X is 1, then it is represented as X'. For $x = 0$, $y = 0$, and $z = 0$, the sum term is written as $x + y + z$ and for $x = 0$, $y = 0$, and $z = 1$, the sum term is written as $x + y + z'$. The sum terms are known as maxterms. Maxterms are represented as M0, M1, M2 to M7. Note that M should be written in uppercase.

X	Y	Z	f	Maxterm
0	0	0	0	M0 $\rightarrow X+Y+Z$
0	0	1	1	M1 $\rightarrow X+Y+Z'$
0	1	0	0	M2 $\rightarrow X+Y'+Z$
0	1	1	1	M3 $\rightarrow X+Y'+Z'$
1	0	0	0	M4 $\rightarrow X'+Y+Z$
1	0	1	0	M5 $\rightarrow X'+Y+Z'$
1	1	0	1	M6 $\rightarrow X'+Y'+Z$
1	1	1	1	M7 $\rightarrow X'+Y'+Z'$

Figure 4: Truth table for three variable POS function.

To write the product of sums form of a Boolean expression, first, identify the max terms for which the output is zero. As seen in the truth table, there are four places where the output is zero. The final Boolean expression will contain these four max terms. POS expression is given by

$$f = (X+Y+Z) * (X+Y'+Z) * (X'+Y+Z) * (X'+Y+Z')$$

$$= M0.M2.M4.M5 \square M(0,2,4,5)$$

3.3.3: Boolean Functions: Exercise

There are two ways of expressing Boolean expressions: canonical form and standard form.

Canonical forms express all binary variables in every product or sum term of the Boolean function, whereas the standard form is a simplified version of the canonical form. Following is the procedure for converting expression in standard SOP form to canonical form.

- Examine each term in the given logic function.
- Retain the term, if it is a minterm or else find out the missing variable in the term.
- If the missing variable is X, then multiply the product term with $(X+X')$.
- Multiply all the products and discard the redundant terms.

Exercise 1: Obtain the canonical SOP form of the following function:

$$F(X,Y) = X + Y$$

Solution

$$F(X,Y) = X + Y$$

$$= X(Y+Y') + Y(X + X') \text{ [Multiply missing term with } Y+Y' \text{ and } X+X']$$

$$= XY + XY' + YX + YX'$$

$$= XY + XY' + XY + X'Y$$

$$= XY + XY' + X'Y$$

Following is the procedure for converting expression in standard POS form to canonical form.

- Convert the function into a sum term using the distributive law.
 - Example: $X + X'Y \square (X + X') (X+Y)$
- Identify the missing variable in the sum term. If the missing term is X, then add XX' in the sum term.
- Apply distributive Law.
- Remove those terms which appear more than once.

Exercise 2:

Obtain the canonical POS form of the following function.

$$F = XY + Z$$

- ☐ $(X + Z)(Y + Z)$
- ☐ $(X + Z + YY')(Y + Z + XX')$
- ☐ $(X + Z + Y)(X + Z + Y')(Y + Z + X)(Y + Z + X')$
- ☐ $(X + Y + Z)(X + Y' + Z)(X + Y + Z)(X' + Y + Z)$
- ☐ $(X + Y + Z)(X + Y' + Z)(X' + Y + Z)$